

Agenti logici: calcolo proposizionale

Claudio Gallicchio

a.a. 2025/2026

Riferimenti



AIMA Cap 7

■ 7.5, 7.6

Dimostrazione di teoremi

Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
  inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
            $\alpha$ , la query, una formula della logica proposizionale  
 $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false  
if Vuoto?( $simboli$ ) then  
  if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )  
  else return true // quando  $KB$  è false, restituisce sempre true  
else do  
   $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )  
  return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )  
         and  
         TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

L'algoritmo DPLL

function DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

inputs: *s*, una formula della logica proposizionale

clausole $\leftarrow$ l'insieme di clausole nella rappresentazione CNF di *s*

simboli $\leftarrow$ una lista di tutti i simboli proposizionali in *s*

return DPLL(*clausole*, *simboli*, { })

function DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

if ogni clausola in *clausole* è vera in *modello* **then return** *true*

if qualche clausola in *clausole* è falsa in *modello* **then return** *false*

P, *valore* $\leftarrow$ Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P, *valore* $\leftarrow$ Trova-Clausola-Unitaria(*clausole*, *modello*)

if *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello* $\leftarrow$ {*P* = *valore*})

P $\leftarrow$ Primo(*simboli*); *resto* $\leftarrow$ Resto(*simboli*)

return DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *true*}) **or**

 DPLL(*clausole*, *resto*, *modello* $\leftarrow$ {*P* = *false*})



Miglioramenti di DPLL

- DPLL è completo e termina sempre
- Alcuni miglioramenti:
 - Analisi di componenti (sotto-problemi indipendenti): se le variabili possono essere suddivise in sotto-insiemi disgiunti (senza simboli in comune)
 - ✓ un risolutore può lavorare separatamente su ciascun componente
 - Ordinamento di variabili e valori: scegliere la variabile che compare in più clausole
 - Backtracking intelligente, riavvio casuale e altre ottimizzazioni ...

Metodi locali per SAT: formulazione

- Gli 'stati' sono interpretazioni, assegnamenti **completi**
- L'obiettivo è un assegnamento che soddisfa tutte le clausole (un modello)
- Si parte da un assegnamento casuale
- Ad ogni passo si cambia il valore di un simbolo proposizionale (*flip*)
- Gli stati sono valutati contando il numero di clausole **soddisfatte** (più sono meglio è)

Metodi locali per SAT: algoritmi

- Ci sono molti massimi locali per sfuggire ai quali serve introdurre perturbazioni casuali
 - Hill climbing con riavvio casuale
 - Simulated Annealing
- Molta sperimentazione per trovare il miglior compromesso tra il grado di “avidità” e casualità
- **WALK-SAT** è uno degli algoritmi più semplici ed efficaci



WalkSAT

WalkSAT ad ogni passo

- Sceglie a caso una clausola non ancora soddisfatta
- Individua un simbolo da modificare (*flip*), scegliendo con probabilità p (di solito 0.5) tra:
 - ✓ Scegliere un simbolo a caso (**passo casuale**, *random walk*)
 - ✓ Scegliere quello che rende più clausole soddisfatte (**passo di ottimizzazione**)
- Si arrende dopo un certo numero di *flip* predefinito

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

WalkSat: l'algoritmo

function WalkSAT(*clausole*, *p*, *max_flips*) **returns** un modello o fallimento

inputs: *clausole*, un insieme di clausole della logica proposizionale

p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5

max_flips, numero massimo di inversioni di valore prima di abbandonare

modello $\leftarrow$ un assegnamento casuale di valori di verità ai simboli in *clausole*

for *i* = 1 **to** *max_flips* **do**

if *modello* soddisfa *clausole* **then return** *modello*

clausola $\leftarrow$ una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*

if Random(0, 1) $\leq$ *p* **then** inverti il valore in *modello* di un simbolo scelto
 casualmente in *clausola*

else inverti il valore di verità del simbolo in *clausole* che massimizza il numero
 di clausole soddisfatte

return fallimento

WalkSat: l'algoritmo

function WalkSAT(*clause*, *p*, *max_flips*) **returns** un modello o fallimento

inputs: *clause*, un insieme di clausole della logica proposizionale

p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5

max_flips, numero massimo di inversioni di valore prima di abbandonare

modello $\leftarrow$ un assegnamento casuale di valori di verità ai simboli in clausole

for *i* = 1 **to** *max_flips* **do**

if *modello* soddisfa clausole **then return** *modello*

clausola $\leftarrow$ una clausola, falsa in *modello*, scelta casualmente nell'insieme *clause*

if Random(0, 1) $\leq$ *p* **then** inverti il valore in *modello* di un simbolo scelto
 casualmente in *clausola*

else inverti il valore di verità del simbolo in *clause* che massimizza il numero
 di clausole soddisfatte

return fallimento

WalkSat: l'algoritmo

function WalkSAT(*clausole*, *p*, *max_flips*) **returns** un modello o fallimento
inputs: *clausole*, un insieme di clausole della logica proposizionale
p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
max_flips, numero massimo di inversioni di valore prima di abbandonare

modello $\leftarrow$ un assegnamento casuale di valori di verità ai simboli in *clausole*

for *i* = 1 **to** *max_flips* **do**
 if *modello* soddisfa *clausole* **then return** *modello*
 clausola $\leftarrow$ una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*
 if Random(0, 1) $\leq$ *p* **then** inverti il valore in *modello* di un simbolo scelto casualmente in *clausola*
 else inverti il valore di verità del simbolo in *clausole* che massimizza il numero di clausole soddisfatte

return fallimento

WalkSat: l'algoritmo

function WalkSAT(*clausole*, *p*, *max_flips*) **returns** un modello o fallimento
inputs: *clausole*, un insieme di clausole della logica proposizionale
p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
max_flips, numero massimo di inversioni di valore prima di abbandonare

modello $\leftarrow$ un assegnamento casuale di valori di verità ai simboli in *clausole*

for *i* = 1 **to** *max_flips* **do**
 if *modello* soddisfa *clausole* **then return** *modello*
 clausola $\leftarrow$ una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*
 if Random(0, 1) $\leq$ *p* **then** inverti il valore in *modello* di un simbolo scelto casualmente in *clausola*
 else inverti il valore di verità del simbolo in *clausole* che massimizza il numero di clausole soddisfatte

return fallimento

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola falsa in modello scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

con probabilità *p*: random walk

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

con probabilità (1-p): ottimizzazione

WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

WalkSAT: un esempio

Come euristica usiamo il numero di clausole soddisfatte (da massimizzare)

Rosso: passo casuale

Verde: passo di ottimizzazione

$$\underbrace{\{\neg B_{1,1}, P_{1,2}, P_{2,1}\}}_1 \underbrace{\{\neg P_{1,2}, B_{1,1}\}}_2 \underbrace{\{\neg P_{2,1}, B_{1,1}\}}_3 \underbrace{\{\neg B_{1,1}\}}_4$$

[$B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$] 2, 3 F; scelgo 2; a caso flip $B_{1,1}$

[$B_{1,1}=T, P_{1,2}=T, P_{2,1}=T$] 4 F; scelgo 4; flip $B_{1,1}$ unica scelta

[$B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$] 2, 3 F; scelgo 2; a caso flip $P_{1,2}$

[$B_{1,1}=F, P_{1,2}=F, P_{2,1}=T$] 3 F; scelgo 3; ottimizzazione: flip $P_{2,1}$ [#4]; flip $B_{1,1}$ [#3]

[$B_{1,1}=F, P_{1,2}=F, P_{2,1}=F$] ho trovato un modello!!!

Analisi di WalkSAT

- Se $max-flips = \infty$ e l'insieme di clausole è soddisfacibile prima o poi termina.
- Ma se vogliamo un algoritmo che termina, questo è necessariamente incompleto
- Va bene per cercare un modello, sapendo che c'è, ma se è insoddisfacibile non termina
- Non può essere usato per verificare l'insoddisfacibilità

Problemi SAT difficili

*cioè con un numero relativamente
piccolo di clausole*

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)
 $\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$
 - Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

Problemi SAT difficili

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)

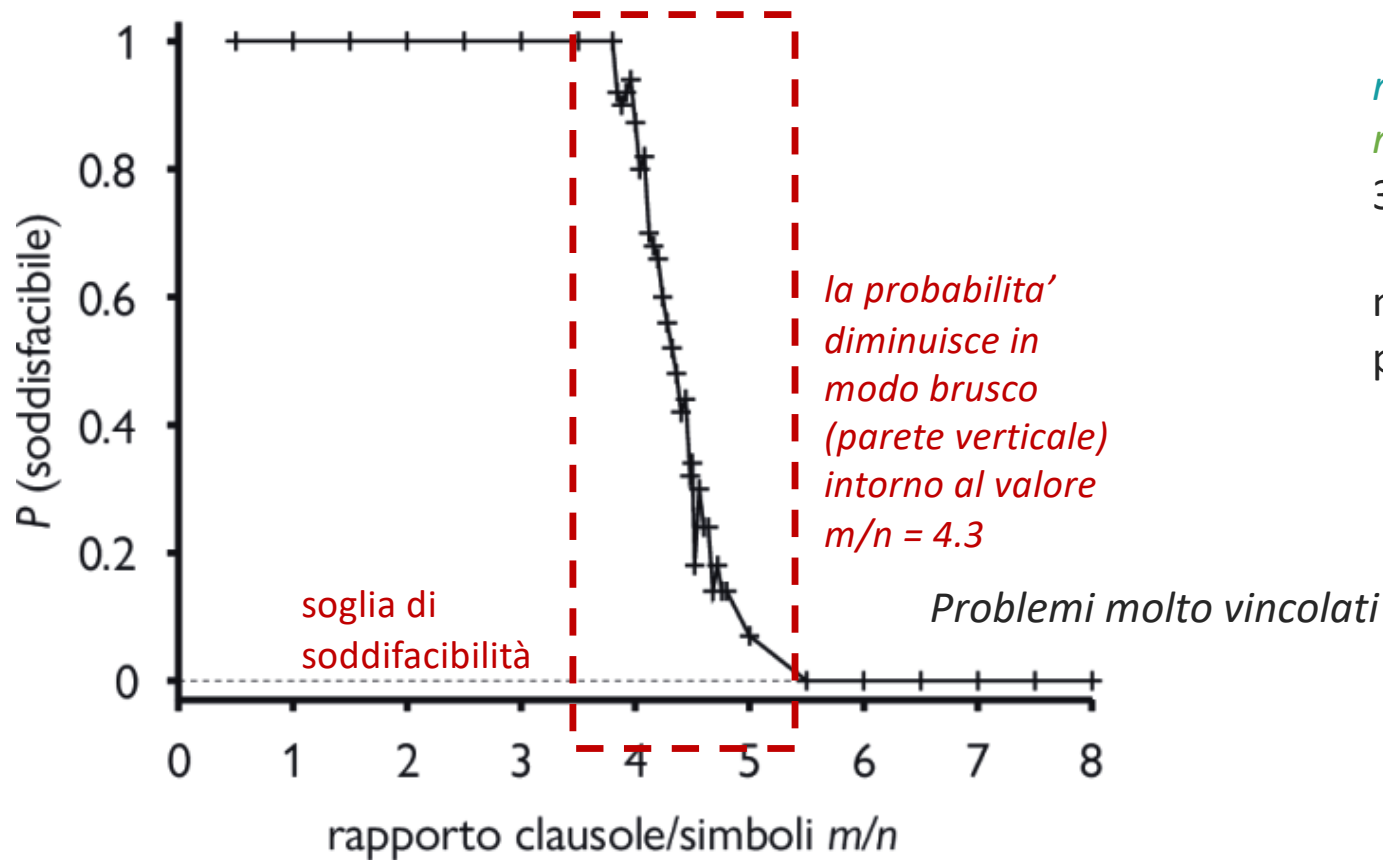
$\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$

- Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

- È significativo il rapporto m/n
 - m è il numero di clausole (vincoli)
 - n il numero di simboli
 - Nell'esempio: $5/5=1$
- Più grande il rapporto, più vincolato è il problema

Probabilità di soddisfacibilità in funzione di m/n

Problemi poco vincolati



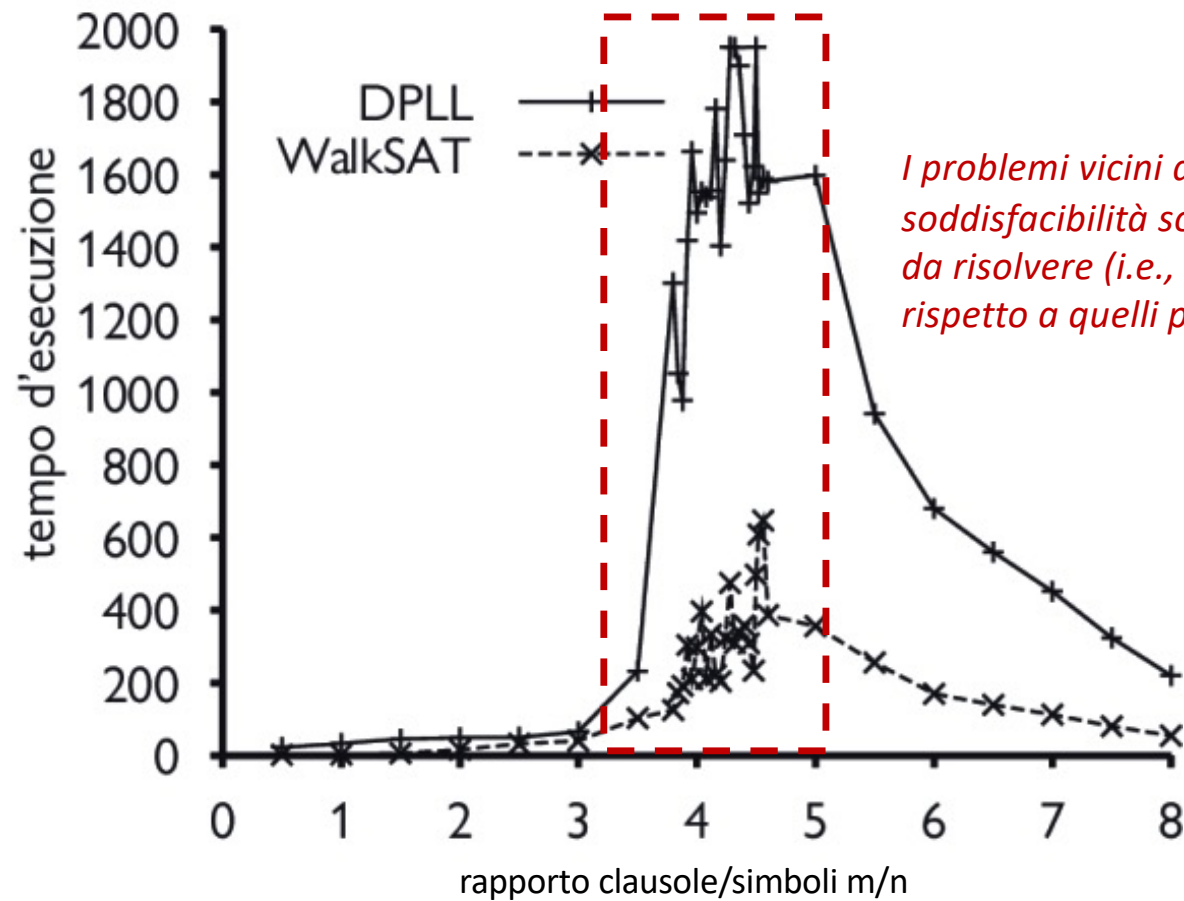
m (n. clausole) varia
 n (n. simboli) = 50
3 letterali per clausola

media su 100
problemi generati a caso

Confronto su problemi soddisfacibili, ma difficili

Confronto tra DPLL e WalkSAT

tempo medio di
esecuzione (misurato in
numero di iterazioni)
per DPLL e WalkSAT su
formule casuali 3-CNF





Inferenza come deduzione

- Un altro modo per decidere se $KB \models A$ è usare un sistema di deduzione
 - Denotiamo con $KB \vdash A$ il fatto che A è derivabile (deducibile) da KB
 - La deduzione avviene specificando delle regole di inferenza.
- In un sistema di inferenza le regole ...
 - dovrebbero derivare solo formule che sono conseguenza logica
 - dovrebbero derivare tutte le formule che sono conseguenza logica

Correttezza e completezza

- **Correttezza:** Se $KB \vdash \alpha$ allora $KB \models \alpha$
 - Tutto ciò che è derivabile è conseguenza logica.
 - Le regole preservano la verità
- **Completezza:** Se $KB \models \alpha$ allora $KB \vdash \alpha$
 - Tutto ciò che è conseguenza logica è ottenibile tramite il sistema deduttivo.

Regole deduttive

- Consentono di **inferire nuove** proposizioni (conclusioni) a partire da proposizioni esistenti (premesse)
- Rappresentate in forma schematica (regola di inferenza)

Premesse

Conclusione

Alcune regole di inferenza per Prop

Le regole sono schemi deduttivi del tipo:

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta} \quad \textit{Modus ponens}$$

$$\frac{\alpha \wedge \beta}{\alpha} \quad \textit{Eliminazione dell'AND}$$

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \textit{Eliminazione e introduzione}$$
$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta} \quad \textit{della doppia implicazione}$$

Una rappresentazione per il Wumpus

$R_1: \neg P_{1,1}$

non ci sono pozzi in [1, 1]

C'è brezza nelle caselle adiacenti ai pozzi:

$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

Percezioni:

$R_4: \neg B_{1,1}$

non c'è brezza in [1, 1]

$R_5: B_{2,1}$

c'è brezza in [2, 1]

$KB = \{R_1, R_2, R_3, R_4, R_5\}$

$KB \models \neg P_{1,2}?$

Dimostrazione

$R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$	$(R_2, \Leftrightarrow E)$
$R_7: (P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$	$(R_6, \wedge E)$
$R_8: \neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$	$(R_7, \text{contrapposizione})$
$R_9: \neg(P_{1,2} \vee P_{2,1})$	$(R_4 \text{ e } R_8, \text{Modus Ponens})$
$R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$	$(R_9, \text{De Morgan})$
$R_{11}: \neg P_{1,2}$	$(R_{10}, \wedge E)$

Dimostrazione come ricerca

Problema: come decidere ad ogni passo qual è la regola di inferenza da applicare? ... e a quali premesse? Come evitare l'esplosione combinatoria?

- Problema di esplorazione di uno spazio di stati
- Una *procedura di dimostrazione* definisce:
 - la *direzione* della ricerca
 - la *strategia* di ricerca

Direzione della ricerca

Nella dimostrazione di teoremi conviene **procedere all'indietro**.

- Con una applicazione *in avanti* delle regole di inferenza non controllata:

Da $A, B: A \wedge B$ $A \wedge (A \wedge B)$... $A \wedge (A \wedge (A \wedge B))$

- Meglio *all'indietro*

- se si vuole dimostrare $A \wedge B$, si cerchi di dimostrare A e poi B
- se si vuole dimostrare $A \Rightarrow B$, si assuma A e si cerchi di dimostrare B
- ...

Strategia di ricerca

■ Completezza

- Le regole della deduzione naturale sono un insieme di regole di inferenza completo (2 per ogni connettivo)
- Se l'algoritmo di ricerca è completo siamo a posto

■ Efficienza

- La complessità è alta: è un problema decidibile ma NP-completo



Regola di risoluzione per PROP

- Meno regole sono meglio è, senza rinunciare alla completezza.
- Un'unica regola: la **regola di risoluzione** (presuppone forma a clausole)

$$\frac{\{P, Q\} \quad \{\neg P, R\}}{\{Q, R\}}$$

$$\frac{(P \vee Q) \wedge (\neg P \vee R)}{(Q \vee R)}$$

- Corretta? Basta pensare ai modelli
- Preferita la notazione insiemistica: gli eventuali duplicati si eliminano

La regola di risoluzione in generale

$$\frac{\{l_1, l_2, \dots, l_i, \dots, l_k\} \quad \{m_1, m_2, \dots, m_j, \dots, m_n\}}{\{l_1, l_2, \dots, l_{i-1}, l_{i+1}, \dots, l_k, m_1, m_2, \dots, m_{j-1}, m_{j+1}, \dots, m_n\}}$$

Gli l e m sono **letterali**, simboli di proposizione positivi o negativi;
 l_i e m_j sono uguali e di segno opposto

Caso particolare

$$\frac{\{P\} \quad \{\neg P\}}{\{\}}$$

*clausola vuota
contraddizione*

Il grafo di risoluzione

c'è brezza in [1,1] sse c'è un pozzo in una delle caselle adiacenti

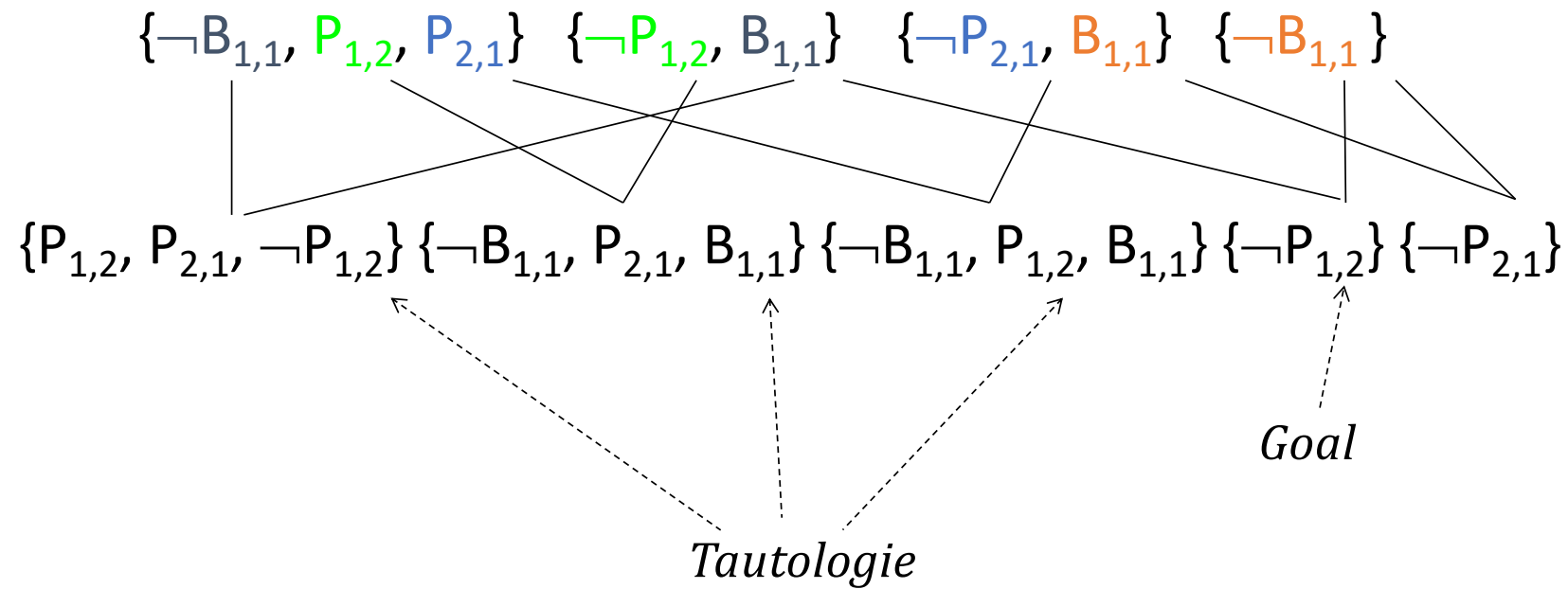
$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\}$ $\{\neg P_{1,2}, B_{1,1}\}$ $\{\neg P_{2,1}, B_{1,1}\}$ $\{\neg B_{1,1}\}$

percezione

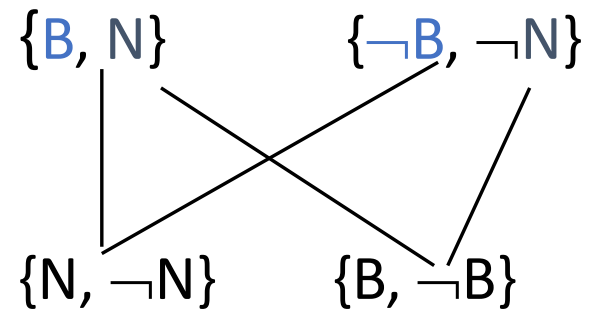
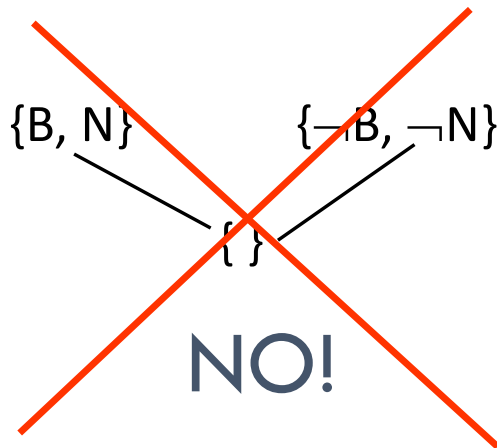
Il grafo di risoluzione

c'è brezza in [1,1] sse c'è un pozzo in una delle caselle adiacenti

percezione



Attenzione!



... e qui ci fermiamo

Un passo alla volta !!!

Ma siamo sicuri che basti una regola? È completo?

Completezza: se $KB \models \alpha$ allora $KB \vdash_{\text{res}} \alpha$?

Non sempre. Ci basta un contro-esempio.

Es. $KB \models \{A, \neg A\}$ ma non è vero che $KB \vdash_{\text{res}} \{A, \neg A\}$

Ma siamo sicuri che basti una regola? È completo?

- Teorema di **refutazione** offre un modo alternativo di procedere:

$KB \models \alpha$ sse $(KB \cup \{\neg\alpha\})$ è insoddisfacibile

- Teorema di risoluzione [ground]:

KB insoddisfacibile sse $KB \vdash_{\text{res}} \{ \}$

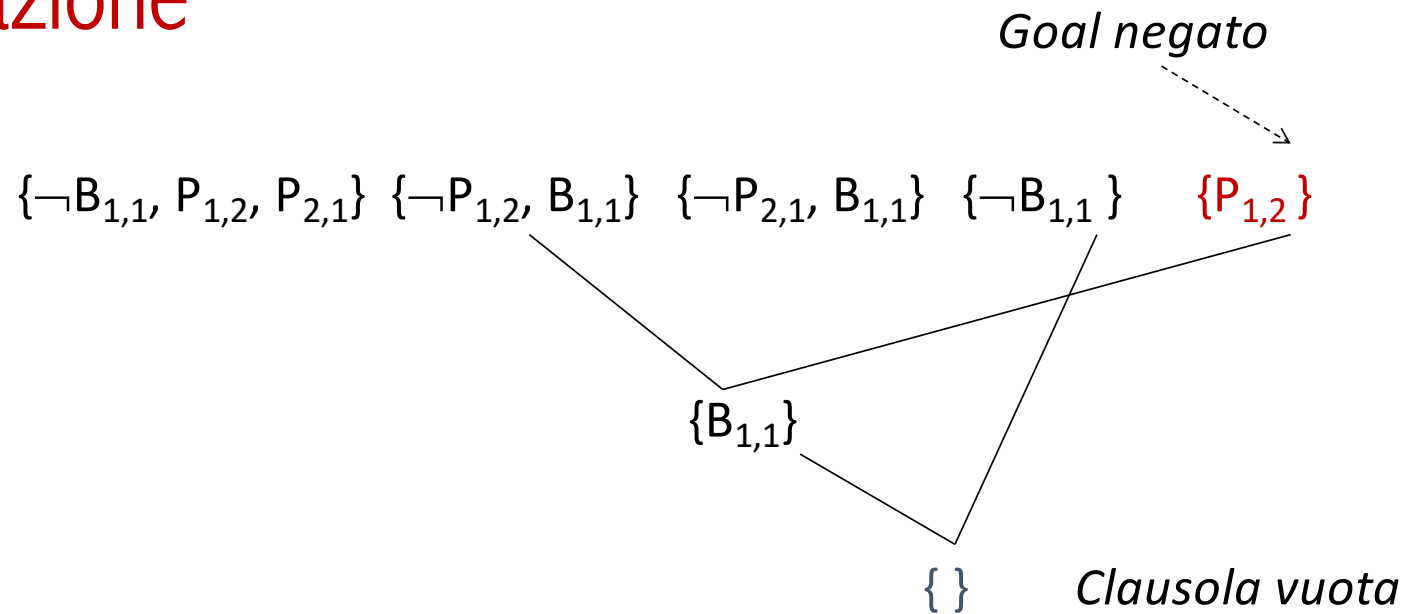
- Nell'esempio:

$KB \cup FC(\neg(A \vee \neg A))$ è insoddisfacibile? Sì, perché ...

$KB \cup \{A\} \cup \{\neg A\} \vdash_{\text{res}} \{ \}$ in un passo

Quindi possiamo concludere che $KB \models \{A, \neg A\}$

Refutazione



Conclusioni

- Abbiamo visto come gli agenti KB che usano PROP come linguaggio di rappresentazione possono decidere se $KB \models \alpha$
- Il problema è decidibile, ma intrattabile (NP) nel caso peggiore
- Esistono algoritmi efficienti e completi che consentono di affrontare problemi di grosse dimensioni
- I metodi locali sono particolarmente efficienti ma non completi